



Alien Code

Assistente de codificação local com harness opencode,
ambiente efêmero e execução visível passo a passo

Sucessor de poc-websocket-demo (assistente) + coding-agent (agente codificador)

Versão	0.1 (rascunho para discussão)
Data	28/09/2026
Referências	awslabs/aidlc-workflows (MIT-0) · Graphify-Labs/graphify (Apache-2.0)
Stack	Java 21 · Spring Boot · Spring AI · React + TypeScript · Docker · opencode · Ollama

Sumário

Sumário	2
1. Resumo executivo	4
1.1 O que muda em relação à V1	4
2. Contexto e problema	5
2.1 Objetivos	5
2.2 Fora de escopo (nesta versão)	6
3. Projetos de referência e como usamos	7
3.1 AI-DLC Workflows (awslabs/aidlc-workflows)	7
3.2 graphify (Graphify-Labs/graphify)	8
3.3 opencode como harness	8
4. Arquitetura	9
4.1 Componentes	9
4.2 Organização do Alien Server (Clean Architecture)	10
5. Ambiente efêmero — a Toca	11
5.1 Modos de semeadura	11
5.2 Imagem alien/toca	11
5.3 Isolamento e limites	12
6. Harness de execução	13
6.1 Loop gerar → verificar → aprender	13
6.2 Condições de parada (halting)	13
6.3 Configuração do opencode na Toca	13
7. Mapa de dependências e DAG de tarefas	15
7.1 Algoritmo do planejador	15
7.2 Execução do DAG	15
8. Execução visível passo a passo	16
8.1 Envelope do evento	16
8.2 Catálogo de eventos	16
8.3 Mapeamento a partir do opencode	17
8.4 Interface	18
9. Fluxo principal e estados	19
9.1 Diagrama de sequência	19
9.2 Máquina de estados da missão	20
9.3 Estados da tarefa	20
10. Modelo de domínio e API	21
10.1 Entidades	21
10.2 API REST	21

10.3 WebSocket 21

11. Requisitos 22

11.1 Funcionais 22

11.2 Não funcionais 22

12. Segurança 23

13. Estrutura do projeto e stack 24

14. Plano de entrega 26

15. Riscos, decisões e glossário 27

15.1 Riscos 27

15.2 Decisões em aberto 27

15.3 Glossário 27

1. Resumo executivo

O **Alien Code** é a segunda versão do estudo de agentes de codificação. Ele junta, num único produto que roda na máquina do desenvolvedor, os dois papéis que na V1 estavam separados: o **assistente** (poc-websocket-demo, chat com streaming) e o **agente codificador** (coding-agent, executado dentro de uma pipeline do GitHub Actions).

Em vez de disparar uma pipeline por tarefa, o Alien Code sobe um **ambiente efêmero local** — um container Docker que chamamos de **Toca** — com os repositórios envolvidos, e dentro dele executa o **opencode** como harness do agente, usando modelos abertos servidos pelo **Ollama**. Tudo o que o agente faz (planejar, ler arquivos, editar, rodar testes, pedir aprovação) vira um evento que aparece **ao vivo, passo a passo**, numa timeline de execução no estilo do Manus e do Claude Code Web.

Dois projetos de referência apresentados no TDC dão a espinha dorsal:

- **AI-DLC Workflows** (awslabs) — metodologia e motor de workflow neutro de harness, com suporte nativo ao opencode: perfis de trabalho, estágios, portões de aprovação humana, sensores de verificação, trilha de auditoria e condições de parada. O Alien Code usa o AI-DLC dentro da Toca e adota seus princípios no orquestrador.
- **graphify** (Graphify-Labs) — transforma o código em um grafo de conhecimento local (tree-sitter, sem LLM). O Alien Code usa o grafo para **descobrir e ordenar dependências entre tarefas**, inclusive entre repositórios diferentes, e o expõe ao agente via MCP para que ele consulte o grafo em vez de varrer arquivos.

A ideia em uma frase: o pedido vira uma **missão**; o grafo de dependências vira um **DAG de tarefas**; a Toca é a **mesa de trabalho descartável**; o opencode é quem **codifica**; a timeline é a **janela** por onde o desenvolvedor acompanha tudo; e o patch aprovado é a **entrega** na branch local.

1.1 O que muda em relação à V1

Aspecto	V1 (poc-websocket-demo + coding-agent)	V2 (Alien Code)
Onde roda o agente	Runner do GitHub Actions, um por issue	Container efêmero local (Toca), um por missão
Harness	Loop próprio em Java (agent-runner)	opencode (servidor HTTP + SSE) + AI-DLC
Dependências entre tarefas	Pipelines independentes, sem visão umas das outras	Grafo global (graphify) + DAG de tarefas na mesma Toca
Visibilidade	Comentário na issue + polling do status	Timeline ao vivo: passos, tools, terminal, diff, grafo
Aprovação humana	Revisão do PR no final	Portões durante a missão (plano, permissões, entrega)
Entrega	PR em rascunho no GitHub	Patch aplicado numa branch local; PR opcional
Infra necessária	Pool de runners, tokens, filas	Docker + Ollama na própria máquina

2. Contexto e problema

A V1 provou o ciclo pensa → age → observa dentro de um container sem rede e a entrega por PR em rascunho. Ao evoluir, apareceram limitações estruturais de executar o agente **em pipeline**:

1. **Pool de máquinas.** Cada tarefa ocupa um runner. Para paralelizar é preciso um pool (self-hosted ou pago), com modelo, caches e imagens aquecidos em cada máquina. Runners padrão são lentos para inferência local.
2. **Dependências cruzadas.** Uma mudança real costuma tocar mais de um repositório (biblioteca → API → front). Em pipelines separadas, a tarefa do front não enxerga a mudança da API que ainda não foi mergeada: cada execução parte de um main diferente e não sabe de quem depende.
3. **Feedback lento e opaco.** O desenvolvedor só vê o resultado no fim (ou um log bruto). Não há como intervir no meio, responder uma dúvida ou negar um comando.
4. **Credenciais e superfície.** Tokens de GitHub, permissões de workflow e regras anti-loop do GITHUB_TOKEN complicam o fluxo.

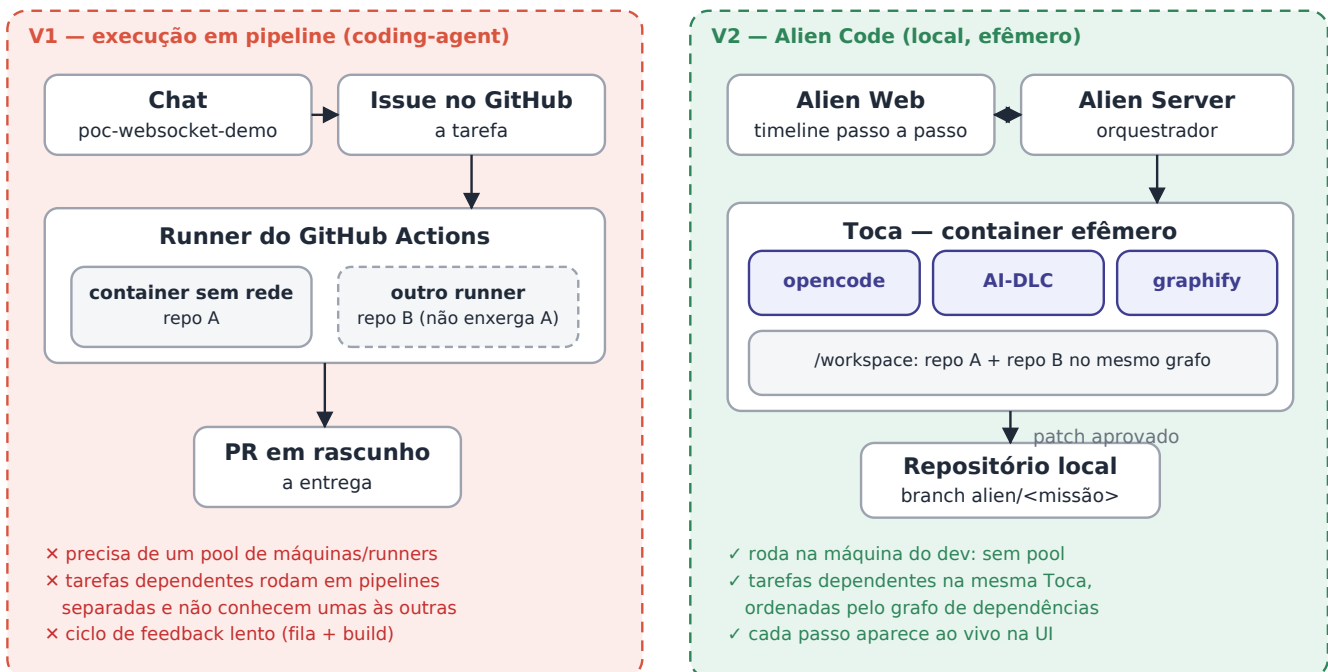


Figura 1. Da execução em pipeline (V1) para a execução local e efêmera (V2).

A resposta da V2 é trazer a execução para perto do desenvolvedor, sem abrir mão do isolamento: a máquina local é o "pool", o container efêmero é o isolamento, e o grafo de dependências é o que permite que tarefas relacionadas sejam executadas **juntas e na ordem certa**.

2.1 Objetivos

- Executar um agente de codificação com modelos abertos (Ollama) **100% local**, sem pipeline.
- Isolar cada missão num **ambiente efêmero** criado a partir de um projeto existente ou de um projeto novo.
- Mostrar **cada passo** da execução em tempo real, com possibilidade de intervir (parar, responder, aprovar, negar).
- Tratar **dependências entre mudanças** (inclusive entre repositórios) de forma explícita, a partir de um grafo.
- Reaproveitar o que o mercado já resolveu: **opencode** como harness, **AI-DLC** como metodologia/motor de workflow e **graphify** como mapa de dependências.
- Continuar sendo material de estudo: arquitetura explícita (Clean Architecture), decisões documentadas, testes.

2.2 Fora de escopo (nesta versão)

- Execução remota ou em nuvem, multiusuário e autenticação corporativa.
- Deploy em produção a partir da missão (a entrega termina na branch local ou num PR).
- Treinar ou ajustar modelos. Usamos modelos abertos como estão.
- Substituir a revisão humana: o Alien Code reduz o esforço de revisão, não o elimina.

3. Projetos de referência e como usamos

3.1 AI-DLC Workflows (awslabs/aidlc-workflows)

O AI-DLC (AI-Driven Development Life Cycle) transforma assistentes de código em workflows de entrega estruturados e verificáveis. Um **núcleo neutro** (core/) é empacotado para vários harnesses — Claude Code, Kiro, Codex, Cursor, Copilot e **opencode** (harness/opencode). Pontos relevantes que encontramos no repositório e na especificação AI-DLC Workflows 2.0:

- **5 fases, 33 estágios, 14 agentes-persona** (arquiteto, desenvolvedor, qualidade, devsecops...) e **11 perfis** de workflow (feature, bugfix, refactor, poc, mvp, security-patch, express...).
- **Modelo de três compartimentos** por estágio: o quê (entradas/saídas), como sabemos que está certo (pós-condições) e o que aprendemos (regras candidatas).
- **Loop autocorretivo com condição de parada**: itera contra pós-condições até convergir ou atingir limite de iterações/tokens; então **escala para o humano**.
- **Portões de aprovação humana**, estado e auditoria controlados por ferramenta (não pelo modelo), e **sensores** (linter, type-check, rastreabilidade, seções obrigatórias).
- No opencode, o AI-DLC instala: opencode.json (skills, instruções e permissões), o comando /aidlc, 14 subagentes em .opencode/agents/ e um **plugin adaptador** que mapeia eventos do opencode (chat.message, tool.execute.before/after, session.idle) para os hooks do núcleo.

Como o Alien Code usa o AI-DLC

O que adotamos	Como	Onde
Motor AI-DLC dentro da Toca	A imagem da Toca traz o binário aidlc; na preparação rodamos aidlc config --harness opencode no workspace da missão.	Toca / passo 3
Perfis como tipo de missão	O orquestrador classifica o pedido (feature, bugfix, refactor, poc, novo projeto → mvp/poc) e escolhe o perfil; tarefas pequenas usam express .	Orquestrador
Portões de aprovação	Perguntas e aprovações que o AI-DLC faz ao humano no opencode são capturadas pelo SSE e viram approval.requested na UI; a resposta volta pela API do opencode.	Hub de eventos / UI
Três compartimentos	Cada tarefa do DAG é descrita como: objetivo e artefatos (o quê), comandos de verificação executáveis (pós-condições) e notas de aprendizado.	Planejador
Halting + escalonamento	Limites de iteração, tokens, tempo e "sem progresso" são aplicados pelo orquestrador, fora do modelo (tool-owned state).	Orquestrador
Trilha de auditoria	aidlc-docs/ (estado, audit) é colhido junto com o patch e anexado à missão.	Entrega
Sem delegação oculta	Só o orquestrador cria sessões/subtarefas; o agente não cria outros agentes livremente.	Princípio de design

Atenção — modelos locais: o AI-DLC recomenda modelos de raciocínio fortes. Com modelos abertos menores, o workflow completo (33 estágios) tende a ser lento e frágil. Por isso o Alien Code tem dois modos: **AI-DLC completo** (perfis maiores, quando o modelo aguenta) e **AI-DLC lite** (perfil express/bugfix ou só os princípios aplicados pelo orquestrador). O modo é configurável por missão.

3.2 graphify (Graphify-Labs/graphify)

O graphify mapeia um projeto (código, docs, PDFs, imagens) em um **grafo de conhecimento** que se consulta em vez de fazer grep. O código é analisado com **tree-sitter** de forma determinística, **sem LLM e sem sair da máquina**. Cada aresta tem um rótulo de confiança (EXTRACTED, INFERRED, AMBIGUOUS). Recursos que usamos diretamente:

- graphify update <dir> — (re)constrói o grafo só por AST; incremental.
- graphify affected "<nó>" --depth N — **raio de impacto**: quem chama, importa, herda ou usa um símbolo (calls, imports, inherits, implements, uses...).
- graphify global add graph.json --as repo e merge-graphs — **grafo entre repositórios**, com passagem que resolve chamadas que cruzam a fronteira de repo (cross_repo_calls).
- graphify --mcp — servidor MCP com ferramentas query_graph, get_node, get_neighbors, shortest_path, god_nodes, graph_stats.
- graphify opencode install — instala AGENTS.md e um plugin tool.execute.before que incentiva o agente a consultar o grafo antes de ler arquivos.
- graph.html, GRAPH_REPORT.md e graph_diff(G_old, G_new) — visualização e comparação antes/depois.

Como o Alien Code usa o graphify

Uso	Detalhe
Descobrir dependências	Ao montar a missão, o planejador pergunta ao grafo quais nós são afetados pelos símbolos citados no pedido e em que repositórios estão.
Ordenar tarefas (DAG)	Arestas calls/imports/uses entre os nós afetados definem a ordem: quem é dependido é feito primeiro; tarefas sem aresta entre si podem rodar em paralelo.
Economizar contexto	Via MCP, o agente pede vizinhos e caminhos em vez de abrir dezenas de arquivos — essencial para modelos locais com janela de 32k tokens.
Testes afetados	Na verificação, affected sobre os arquivos alterados seleciona as classes de teste a rodar primeiro (rápido), antes da suíte completa.
Revisão da entrega	O graph diff (antes × depois) mostra novas dependências, ciclos de import e nós centrais tocados — um resumo estrutural para o revisor.
Visualização	O graph.html da Toca é servido na aba Grafo da UI, com os nós tocados destacados.

Licenças: AI-DLC é MIT-0 e graphify é Apache-2.0; ambos permitem uso e empacotamento na imagem da Toca, mantendo os avisos de licença (NOTICE do graphify).

3.3 opencode como harness

O opencode é o agente que efetivamente codifica. Escolhemos o opencode porque ele (1) funciona com provedores abertos, inclusive Ollama; (2) tem um **modo servidor** (opencode serve, porta padrão 4096) com API HTTP e um fluxo de **Server-Sent Events** em GET /event — exatamente o que precisamos para a timeline; (3) é suportado nativamente pelo AI-DLC e pelo graphify.

Endpoint do opencode	Uso no Alien Code
POST /session	Uma sessão por tarefa do DAG (sessões paralelas para tarefas independentes).
POST /session/:id/prompt_async	Envia a instrução da tarefa sem bloquear; o progresso chega pelo SSE.
GET /event (SSE)	Fonte principal da timeline: partes de mensagem, tool calls, arquivos, permissões, idle.
POST /session/:id/permissions/:pid	Responde aprovações (ex.: comando fora da lista permitida).
POST /session/:id/abort	Botão Parar da UI e condição de parada do orquestrador.
GET /session/:id/diff	Diff da sessão para a aba Diff e para conferência na colheita.

A API é protegida por OPENCODE_SERVER_PASSWORD (basic auth) e só é exposta na rede interna da Toca. A especificação OpenAPI exata deve ser conferida na versão fixada da imagem (endpoint /doc).

4. Arquitetura

O sistema tem três blocos: o **Alien Web** (interface), o **Alien Server** (orquestrador, na máquina do dev) e a **Toca** (container efêmero onde o agente trabalha). O Ollama roda no host e é a única saída de rede permitida à Toca.

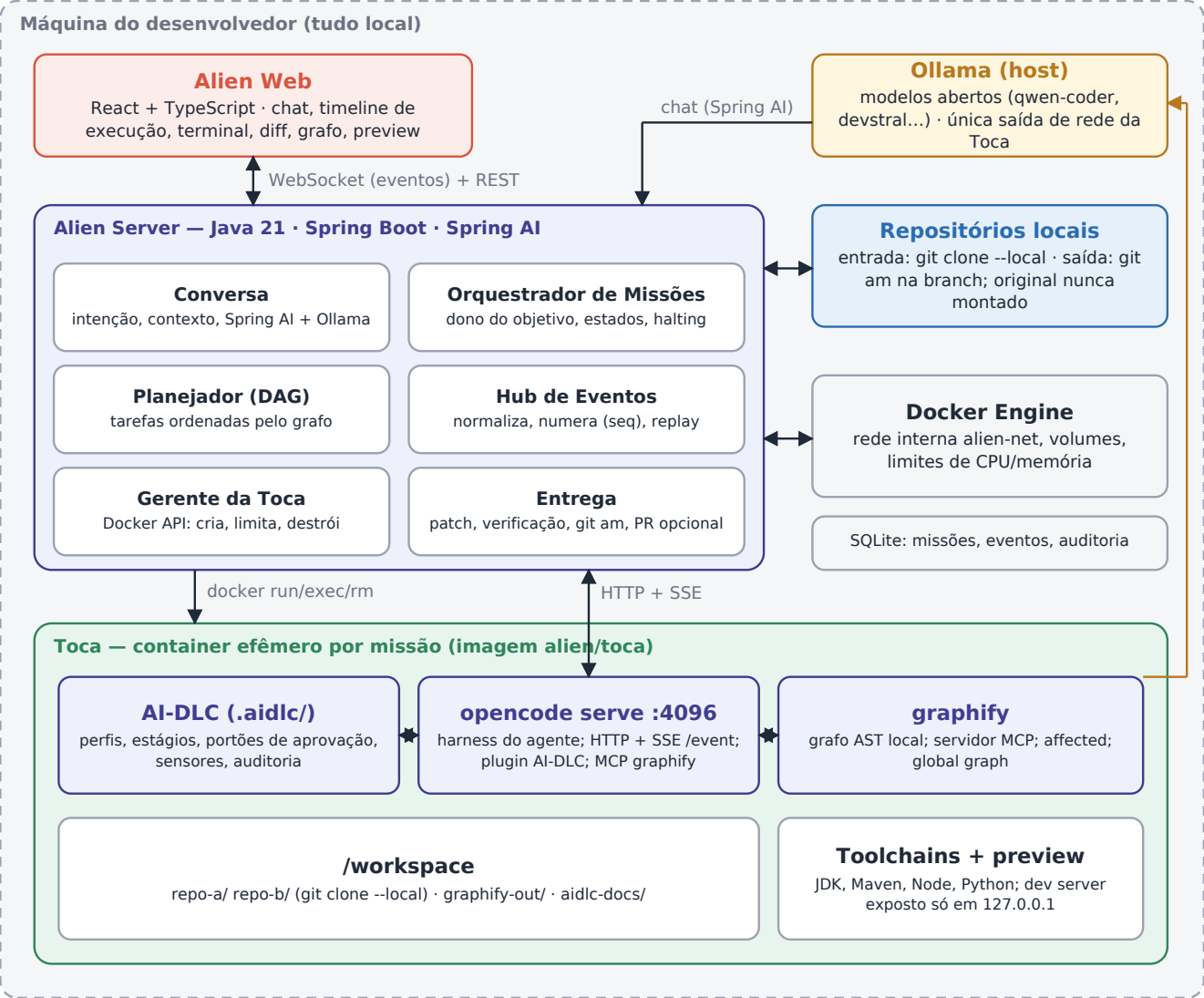


Figura 2. Visão de componentes. Setas duplas indicam comunicação nos dois sentidos.

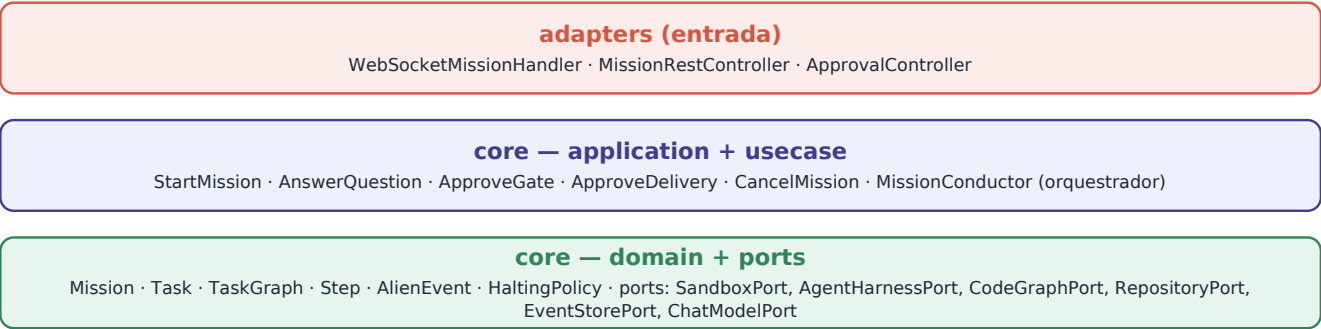
4.1 Componentes

Componente	Responsabilidade	Tecnologia
Alien Web	Chat, timeline de execução, terminal ao vivo, diff, grafo, preview, painel de aprovações.	React 18 + TypeScript + Vite
Conversa	Entende a intenção, faz perguntas de clarificação, decide quando abrir uma missão.	Spring AI + Ollama (streaming, como na V1)
Orquestrador de Missões	Dono do objetivo; máquina de estados; aplica halting; conduz o DAG; registra auditoria.	Java 21, Spring Boot
Planejador (DAG)	Transforma pedido + grafo em tarefas com dependências, contratos e pós-condições.	Java + graphify (CLI)
Hub de Eventos	Recebe eventos de todas as fontes, normaliza, numera e distribui por WebSocket.	Spring WebSocket, SQLite
Gerente da Toca	Cria, limita, observa e destrói containers; volumes; rede interna.	docker-java (Docker Engine API)

Componente	Responsabilidade	Tecnologia
Entrega	Colhe patch, roda verificação final, aplica na branch local após aprovação; PR opcional.	git CLI, gh CLI (opcional)
Toca	Ambiente efêmero com opencode serve, AI-DLC, graphify, toolchains e o workspace.	Imagem Docker alien/toca

4.2 Organização do Alien Server (Clean Architecture)

Mantemos o estilo da V1: portas e adaptadores, um caso de uso por classe, domínio sem dependência de framework. Trocar o opencode por outro harness (ou o Docker por Podman) é trocar um adaptador.



adapters (saída): DockerSandboxAdapter · OpencodeHttpAdapter (HTTP+SSE) · GraphifyCliAdapter · GitCliAdapter · SqliteEventStore · OllamaChatAdapter

Figura 3. Camadas do Alien Server e principais portas.

5. Ambiente efêmero — a Toca

A Toca é um container criado **por missão** e destruído no fim. Ela resolve o isolamento que na V1 vinha do runner do GitHub, mas localmente e em segundos. O repositório original do desenvolvedor **nunca é montado** na Toca: o agente trabalha numa cópia, e só um patch aprovado volta.

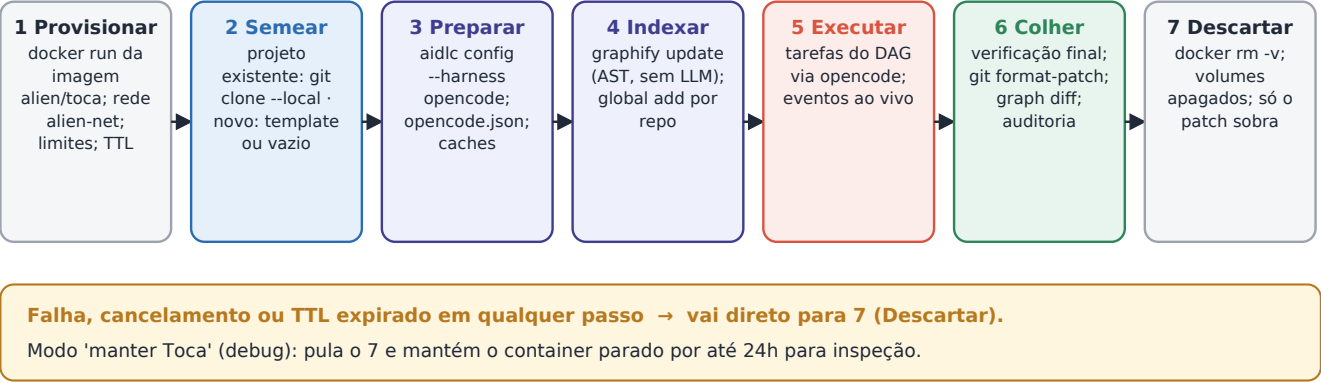


Figura 4. Ciclo de vida da Toca.

5.1 Modos de semeadura

Modo	Como a Toca é semeada	Observações
Projeto existente	git clone --local de cada repo selecionado para /workspace/<repo>, na branch/commit escolhidos; alterações não commitadas podem ser incluídas como patch inicial (opcional).	Um ou mais repositórios na mesma Toca (multi-repo).
Projeto novo	Template do catálogo (spring-boot-api, react-vite, python-cli...) ou diretório vazio; git init dentro da Toca.	Perfil AI-DLC mvp/poc; a entrega cria um novo diretório local em vez de um patch.
Continuar missão	Reaplica o patch da missão anterior sobre o mesmo commit base.	Ajustes após revisão (equivalente ao "ajustes pelo PR" da V1).

5.2 Imagem alien/toca

Dockerfile (esboço) — versões sempre fixadas, imagem construída uma vez e reutilizada

```
FROM eclipse-temurin:21-jdk # base com JDK; Node e Python adicionados abaixo
RUN apt-get install -y git curl ripgrep python3 python3-venv nodejs npm maven
RUN npm i -g opencode-ai@<versão-fixada> # harness
RUN curl -fsSL .../aidlc-workflows/releases/<versão>/install.sh | sh # AI-DLC
RUN pipx install graphify==<versão-fixada> # graphify (pacote com dois y)
RUN useradd -m -u 1000 alien
USER alien
WORKDIR /workspace
COPY toca-entrypoint.sh /usr/local/bin/
ENTRYPOINT ["toca-entrypoint.sh"] # sobe: opencode serve --hostname 0.0.0.0 --port 4096
```

5.3 Isolamento e limites

Controle	Valor padrão	Motivo
Rede	rede Docker internal alien-net; saída só para o proxy do Ollama e, se habilitado, para um proxy de pacotes com lista de domínios (Maven Central, npm)	Mesmo princípio "sem rede" da V1, com exceção controlada
Usuário	não-root (uid 1000), --cap-drop ALL, no-new-privileges	Reduz impacto de comando malicioso
Recursos	4 CPUs, 6 GB RAM, 256 PIDs, disco do volume 10 GB	Não travar a máquina do dev
Tempo de vida	TTL de 60 min por missão (renovável pela UI)	Evitar containers esquecidos
Credenciais	nenhuma: sem SSH, sem token Git, sem ~/.m2/settings com senha	A entrega é feita pelo host, não pela Toca
Portas	4096 (opencode) e porta de preview publicadas só em 127.0.0.1	Nada exposto na rede local
Caches	volumes somente leitura de ~/.m2 e cache npm do host (opcional)	Build rápido sem rede

Equivalente em linha de comando do que o Gerente da Toca faz via Docker API

```
docker run -d --name toca-m42 --network alien-net \
  --cpus 4 --memory 6g --pids-limit 256 --cap-drop ALL \
  --security-opt no-new-privileges --user 1000:1000 \
  -v toca-m42-ws:/workspace -v ~/.m2/repository:/home/alien/.m2/repository:ro \
  -p 127.0.0.1:4096 -e OPENCODE_SERVER_PASSWORD=<aleatória> \
  --label alien.mission=42 --label alien.ttl=3600 alien/toca:0.1
```

6. Harness de execução

O Alien Code não reimplementa o loop do agente (como fazia o agent-runner da V1): ele **conduz** o opencode. O orquestrador decide o quê e em que ordem; o opencode decide como codificar cada tarefa; o AI-DLC dá a estrutura de estágios, portões e verificação.

6.1 Loop gerar → verificar → aprender

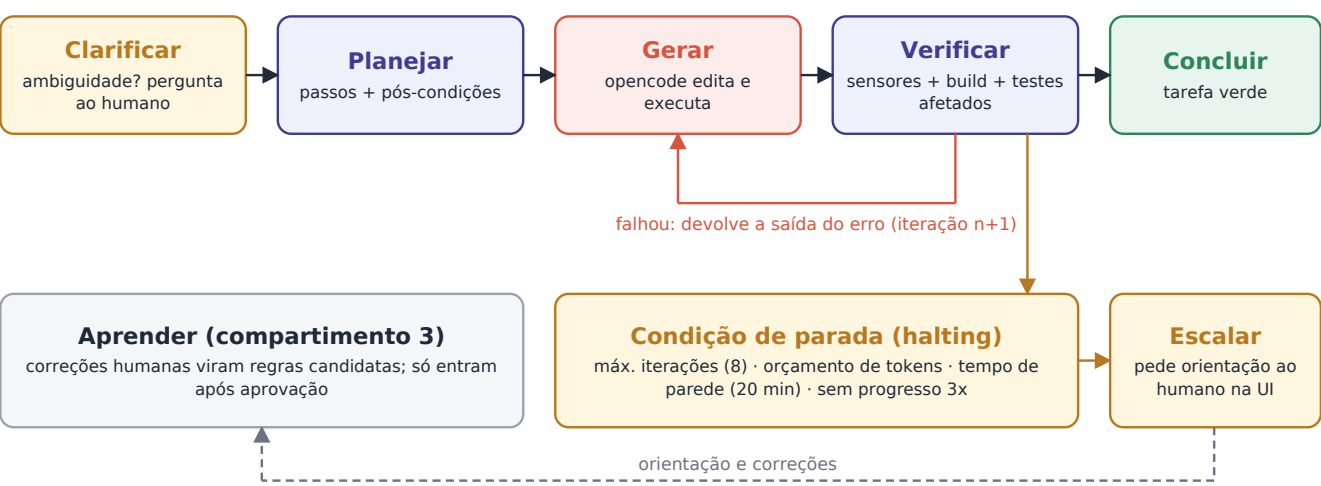


Figura 5. Loop autocorretivo por tarefa, com condição de parada e escalonamento (inspirado no AI-DLC 2.0).

A verificação de cada tarefa é **executável e fora do alcance do modelo** (princípio 3 do AI-DLC): os comandos de pós-condição são definidos no plano e rodados pelo orquestrador via docker `exec`, não pelo agente. O agente pode rodar testes para se orientar, mas o veredito vem do orquestrador — como na V1, em que a pipeline verificava por conta própria.

6.2 Condições de parada (halting)

Condição	Padrão	Ao atingir
Iterações de verificação por tarefa	8	Escala: mostra último erro e pede orientação
Orçamento de tokens por missão	400k (somado do opencode)	Pausa a missão; pede aprovação para continuar
Tempo de parede por tarefa	20 min	Aborta a sessão (/abort) e escala
Sem progresso	3 iterações com o mesmo erro e diff igual	Escala imediatamente
TTL da Toca	60 min	Aviso aos 50 min; descarte ao expirar

6.3 Configuração do opencode na Toca

opencode.json gerado na preparação (mesclado com o que o aidlc config instala)

```
{
  "$schema": "https://opencode.ai/config.json",
  "provider": {
    "ollama": {
      "npm": "@ai-sdk/openai-compatible",
      "options": { "baseUrl": "http://ollama-proxy:11434/v1" },
      "models": { "qwen3-coder:30b": {}, "devstral:24b": {} }
    }
  },
  "model": "ollama/qwen3-coder:30b",
  "mcp": {
    "graphify": { "type": "local", "command": ["graphify", "--mcp", "/workspace"] }
  },
  "permission": {
    "edit": "allow",
    "bash": {
      "*": "ask",
      "mvn *": "allow",
      "npm test*": "allow",
      "git status": "allow",
      "git diff*": "allow",
      "graphify *": "allow"
    }
  }
}
```

Comandos marcados como ask viram pedidos de aprovação na UI. Modelos citados são exemplos; o modelo é escolhido por missão entre os disponíveis no Ollama.

7. Mapa de dependências e DAG de tarefas

Este é o ponto que a V1 não resolvia: **tarefas que dependem umas das outras**. No Alien Code, todas as tarefas de uma missão rodam **na mesma Toca**, sobre o mesmo workspace, numa ordem derivada do grafo.

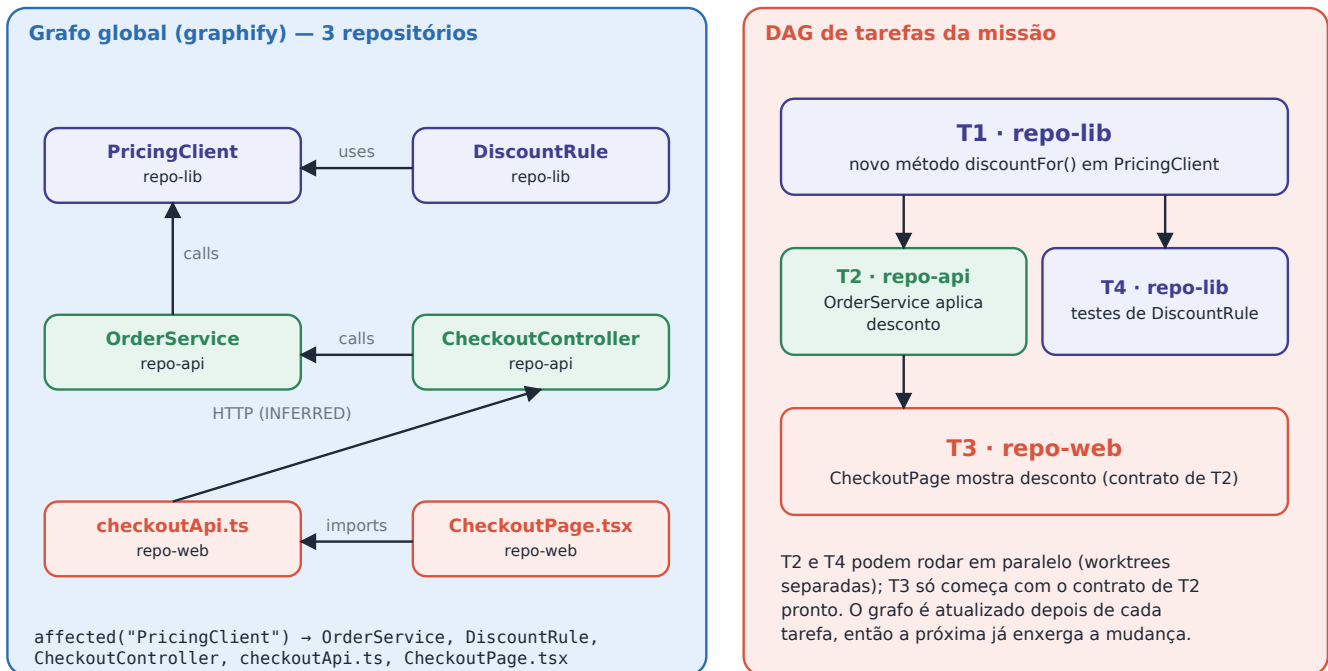


Figura 6. Do grafo global de três repositórios ao DAG de tarefas da missão.

7.1 Algoritmo do planejador

1. **Indexar**: `graphify update` em cada repo e `graphify global add` para compor o grafo global da missão.
2. **Ancorar**: o modelo de conversa extrai do pedido os conceitos citados; o planejador os resolve para nós do grafo (`query_graph`). Âncoras ambíguas viram pergunta ao usuário.
3. **Expandir**: `affected` (profundidade 2, configurável) a partir das âncoras gera o conjunto de nós impactados, agrupados por repositório e módulo.
4. **Propor tarefas**: o modelo agrupa nós impactados em tarefas coesas (normalmente uma por módulo), cada uma com objetivo, arquivos prováveis, **contrato** que produz/consume e comandos de verificação.
5. **Ordenar**: se algum nó da tarefa B depende (`calls/imports/uses`) de um nó da tarefa A, cria-se a aresta $A \rightarrow B$. Ciclos são quebrados juntando as tarefas. O resultado é um DAG com ordenação topológica.
6. **Aprovar**: o DAG é mostrado ao usuário (aba Plano) antes de qualquer execução.

7.2 Execução do DAG

- Tarefas cujo predecessor terminou verde ficam **prontas**. Até N (padrão 2) rodam em paralelo, cada uma em uma sessão `opencode` e numa **git worktree** própria dentro da Toca.
- Ao terminar uma tarefa, sua `worktree` é integrada ao workspace principal (merge rápido; conflito → nova tarefa de integração ou escalonamento), e o grafo é **atualizado incrementalmente**. Assim, a tarefa seguinte já enxerga a mudança real da anterior — o que resolve o problema das pipelines separadas.
- O **contrato** de uma tarefa (assinatura pública, endpoint, DTO) é passado no prompt das dependentes, junto com o trecho do grafo relevante.
- Uma tarefa que escala ao humano bloqueia apenas seus descendentes; ramos independentes continuam.

8. Execução visível passo a passo

Requisito central: o desenvolvedor acompanha **tudo** enquanto acontece, como no Manus ou no Claude Code Web — e pode interferir. Para isso, toda fonte de informação vira um evento com o mesmo envelope, armazenado em ordem e enviado por WebSocket.



Figura 7. Caminho dos eventos até a interface.

8.1 Envelope do evento

Exemplo: um comando de teste que falhou dentro da tarefa T2

```
{
  "v": 1,
  "missionId": "m-42",
  "seq": 318,
  "ts": "2026-09-28T14:03:11.482Z",
  "type": "tool.completed",
  "stepId": "t2.tool.17",
  "parentStepId": "t2",
  "source": "opencode",
  "payload": {
    "tool": "bash", "title": "mvn -q test -pl api",
    "status": "failed", "exitCode": 1, "durationMs": 14210,
    "outputRef": "blob:m-42/318"
  }
}
```

8.2 Catálogo de eventos

Tipo	Quando	Na UI
mission.created / mission.state	Missão aberta; mudança de estado	Cabeçalho, badge de estado
plan.proposed / plan.updated	DAG proposto ou alterado	Aba Plano; passos pendentes na timeline
step.started / step.completed / step.failed	Início e fim de qualquer passo	Linha da timeline com ícone e duração
assistant.delta / thinking.delta	Texto e raciocínio do agente em streaming	Texto sob o passo (raciocínio recolhível)
tool.started / tool.completed	Tool call do opencode (read, edit, bash, MCP...)	Sub-linha ↵ com argumentos e resultado
terminal.output	stdout/stderr de comandos	Aba Terminal ao vivo
file.changed / diff.updated	Arquivo editado; diff consolidado	Aba Diff com +/− por arquivo
graph.updated	Grafo reindexado; nós tocados	Aba Grafo com destaque

Tipo	Quando	Na UI
approval.requested / approval.resolved	Portão AI-DLC, permissão do opencode, plano, entrega	Cartão de aprovação
question.asked / question.answered	Agente precisa de informação	Pergunta no chat
verification.result	Pós-condições avaliadas	Selo verde/vermelho na tarefa
budget.updated	Tokens, tempo, iterações	Contadores no cabeçalho
preview.ready	Dev server da Toca respondeu	Aba Preview (iframe)
delivery.ready / delivery.applied	Patch pronto; aplicado na branch	Resumo final e link da branch/PR

8.3 Mapeamento a partir do opencode

Evento SSE do opencode	Evento Alien
message.part.updated (parte text)	assistant.delta
message.part.updated (parte reasoning)	thinking.delta
message.part.updated (parte tool: pending/running)	tool.started
message.part.updated (parte tool: completed/error)	tool.completed (+ terminal.output para bash)
file.edited / session.diff	file.changed / diff.updated
permission.* (pedido de permissão)	approval.requested
todo.updated	plan.updated (sub-passos da tarefa)
session.idle	gatilho da verificação da tarefa
session.error	step.failed

Nomes de eventos devem ser validados contra a versão fixada do opencode; o adaptador isola essas diferenças do restante do sistema (mesma estratégia do plugin adaptador do AI-DLC).

8.4 Interface

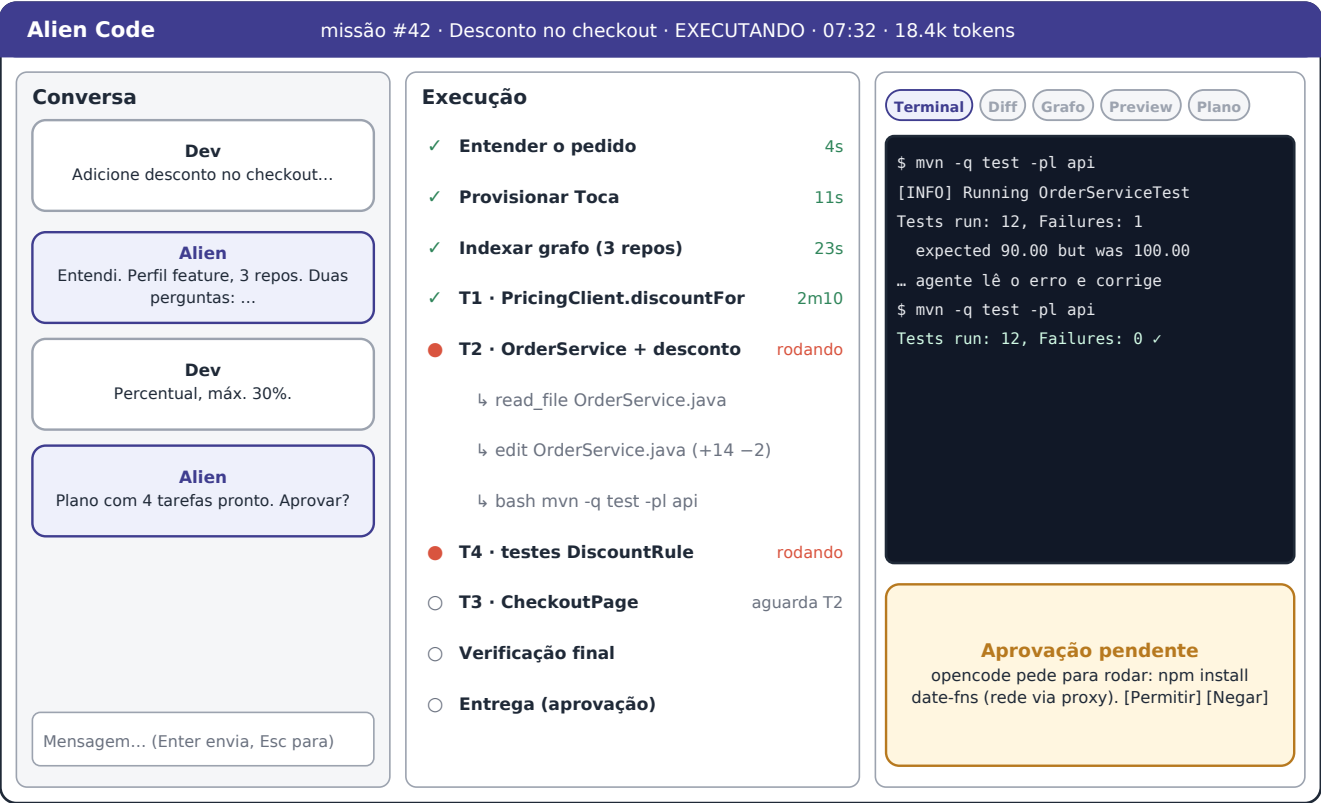


Figura 8. Wireframe da tela da missão: conversa, timeline de execução e painel com abas.

- **Timeline:** uma linha por passo (✓ concluído, ● rodando, ○ pendente, ✕ falhou), com duração; tool calls aparecem como sub-linhas recolhíveis. Clicar num passo foca o terminal/diff daquele passo.
- **Intervenções:** Parar (Esc) aborta a sessão atual; mandar mensagem durante a execução adiciona orientação à tarefa corrente (herdado da V1: interromper e substituir a rodada); aprovar/negar cartões.
- **Reconexão:** ao reabrir a aba, o cliente envia lastSeq e recebe o que perdeu — a missão continua rodando no servidor independente do navegador.
- **Replay:** missões terminadas podem ser reproduzidas a partir do Event Store (útil para estudo e demo).

9. Fluxo principal e estados

9.1 Diagrama de sequência

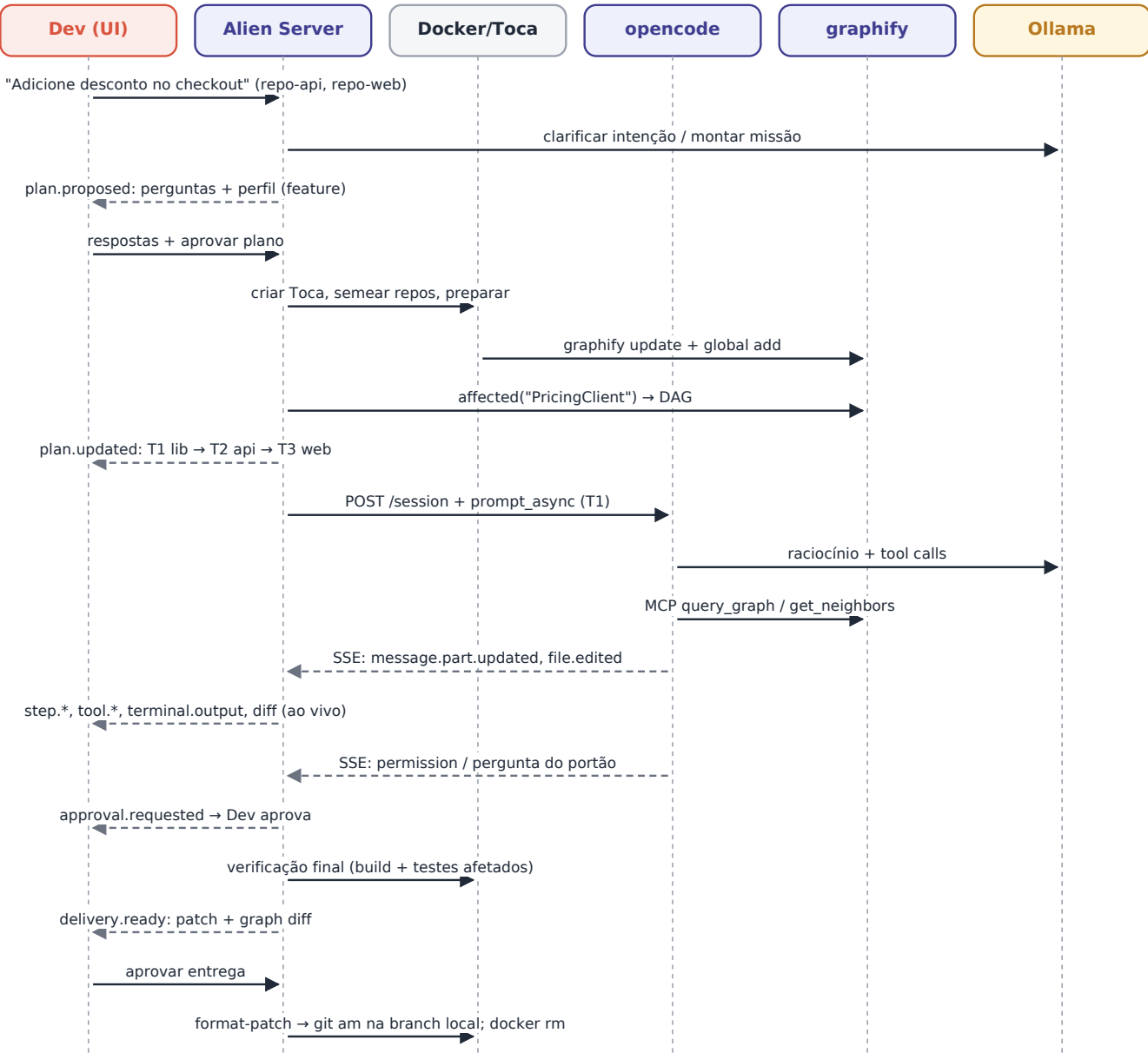


Figura 9. Missão multi-repo do pedido à entrega. Setas tracejadas são eventos/respostas.

9.2 Máquina de estados da missão



Figura 10. Estados da missão. Transições são decididas pelo orquestrador, nunca pelo modelo. FALHOU, CANCELADA e ENTREGUE terminam em DESCARTADA (a Toca é removida).

9.3 Estados da tarefa

Cada tarefa do DAG tem seu próprio ciclo: **PENDENTE** → **PRONTA** → **EXECUTANDO** ⇄ **VERIFICANDO** → **VERDE**, com saídas **AGUARDANDO_HUMANO** (portão, permissão ou pergunta), **ESCALADA** (halting), **BLOQUEADA** (predecessor escalado/falhou) e **CANCELADA**. A missão só vai para VERIFICANDO quando todas as tarefas estão VERDE.

10. Modelo de domínio e API

10.1 Entidades

Entidade	Campos principais
Mission	id, título, pedido original, perfil (feature/bugfix/...), modo AI-DLC (full/lite), estado, repos[], modelo, orçamento, criadaEm, tocalId
RepoRef	caminho local, commit base, branch de entrega, incluir não-committados?
TaskGraph	tarefas[], arestas[] (de → para, motivo: nós e relação do grafo)
Task	id, repo, objetivo, contrato (produz/consome), arquivos prováveis, pós-condições[], estado, sessionId opencode, iterações
PostCondition	comando, diretório, timeout, obrigatória?
Step	id, parentId, tipo, título, estado, início/fim
AlienEvent	missionId, seq, ts, type, stepId, parentStepId, source, payload
Approval	id, tipo (plano, permissão, portão, entrega), pergunta, opções, resposta, quem, quando
Delivery	patches por repo, resumo, graph diff, auditoria AI-DLC, verificação final, branch/PR

10.2 API REST

Método e rota	Descrição
POST /api/missions	Abre missão a partir do chat (pedido, repos ou template, modelo, modo)
GET /api/missions/{id}	Snapshot: estado, plano, orçamento, última seq
POST /api/missions/{id}/plan/approve	Aprova (ou edita e aprova) o DAG
POST /api/missions/{id}/approvals/{aid}	Responde aprovação: allow, deny, allow-always, texto
POST /api/missions/{id}/messages	Orientação do usuário para a tarefa corrente
POST /api/missions/{id}/stop	Para a tarefa corrente (abort) ou a missão inteira
POST /api/missions/{id}/delivery/approve	Aplica patches nas branches locais; opcional: abrir PR
GET /api/missions/{id}/blobs/{ref}	Saídas grandes (logs, diffs) referenciadas por eventos
GET /api/repos · GET /api/templates · GET /api/models	Catálogos para abrir missão

10.3 WebSocket

Canal de eventos e comandos da missão (compatível com o estilo da V1)

```
WS /ws/missions/{id}?lastSeq=317      // servidor → cliente: envelopes AlienEvent (seq > 317)
cliente → servidor:
  {"type":"user_message", "content":"use BigDecimal, não double"}
  {"type":"stop"}
  {"type":"approval", "approvalId":"a-9", "decision":"allow"}
```

11. Requisitos

11.1 Funcionais

ID	Requisito	Prior.
RF-01	Conversar com o Alien em streaming e abrir uma missão a partir da conversa.	Alta
RF-02	Abrir missão sobre um ou mais repositórios locais existentes, escolhendo branch/commit base.	Alta
RF-03	Abrir missão de projeto novo a partir de template ou vazio.	Alta
RF-04	Criar uma Toca isolada por missão e destruí-la ao final, falha, cancelamento ou TTL.	Alta
RF-05	Executar tarefas com opencode + modelo do Ollama dentro da Toca.	Alta
RF-06	Mostrar cada passo ao vivo: estado, raciocínio, tool calls, terminal, diffs.	Alta
RF-07	Permitir parar, orientar, responder perguntas e aprovar/negar permissões durante a execução.	Alta
RF-08	Verificar cada tarefa com pós-condições executáveis fora do controle do modelo.	Alta
RF-09	Aplicar halting (iterações, tokens, tempo, sem progresso) e escalar ao humano.	Alta
RF-10	Entregar patch aprovado em branch local alien/<missão>; PR opcional pelo host.	Alta
RF-11	Construir o grafo (graphify) e expô-lo ao agente via MCP.	Média
RF-12	Gerar DAG de tarefas a partir do grafo, incluindo dependências entre repositórios.	Média
RF-13	Executar tarefas independentes em paralelo (worktrees) e integrá-las.	Média
RF-14	Usar perfis, portões e sensores do AI-DLC (modo full) ou princípios (modo lite).	Média
RF-15	Reconectar sem perder eventos e reproduzir missões antigas (replay).	Média
RF-16	Preview do app rodando na Toca.	Baixa
RF-17	Registrar correções humanas como regras candidatas (aprendizado) com aprovação.	Baixa

11.2 Não funcionais

ID	Requisito
RNF-01	Toca pronta (provisionar + semear + preparar) em até 15 s com imagem já baixada, sem contar a indexação.
RNF-02	Latência evento → tela abaixo de 300 ms no mesmo host.
RNF-03	Nenhuma credencial do desenvolvedor dentro da Toca; repo original nunca montado.
RNF-04	Nenhum container órfão: reconciliador na inicialização remove Tocas sem missão ativa (por label).
RNF-05	Event Store durável: reiniciar o servidor não perde a história da missão.
RNF-06	Tudo funciona offline, exceto download opcional de dependências via proxy com allowlist.
RNF-07	Versões fixadas de opencode, AI-DLC e graphify na imagem; atualização é decisão explícita.
RNF-08	Cobertura de testes do domínio e casos de uso ≥ 80%; testes de integração com Testcontainers.

12. Segurança

O agente executa comandos gerados por um modelo. Tratamos tudo o que sai do modelo como **não confiável**, como na V1 ("o revisor não depende do que o modelo afirmou").

Ameaça	Mitigação
Comando destrutivo ou exfiltração	Toca sem rede (exceto Ollama/proxy com allowlist), não-root, sem capabilities, sem credenciais; bash em modo ask exceto lista permitida.
Prompt injection vindo do código/docs do repo	Permissões do opencode + aprovação humana para comandos fora da lista; conteúdo do repo nunca vira instrução do orquestrador.
Patch malicioso ou com segredos	Verificação final pelo orquestrador; varredura de segredos (gitleaks) no patch; revisão humana obrigatória antes do git am.
Alteração no repo original	Repo original nunca montado; entrega só via git am em branch nova, nunca na branch atual; working tree sujo bloqueia a aplicação.
Exposição da API do opencode	Senha aleatória por Toca; portas publicadas apenas em 127.0.0.1.
Esgotar recursos do host	Limites de CPU/memória/PIDs/disco; TTL; reconciliador de órfãos.
Cadeia de suprimentos da imagem	Versões fixadas, checksums no Dockerfile, imagem construída localmente.

13. Estrutura do projeto e stack

Monorepo proposto

```
alien-code/
├── alien-server/                                Java 21 · Spring Boot 3 · Spring AI (Clean Architecture)
│   └── src/main/java/dev/aliencode/
│       ├── core/mission/{domain,usecase,application}
│       ├── core/planning/                        TaskGraph, planejador, contratos
│       ├── core/events/                         AlienEvent, EventStorePort
│       └── adapters/
│           ├── web/                             REST + WebSocket
│           ├── sandbox/docker/                  DockerSandboxAdapter (docker-java)
│           ├── harness/opencode/                OpencodeHttpAdapter + SSE → AlienEvent
│           ├── graph/graphify/                  GraphifyCliAdapter (docker exec)
│           ├── git/                             GitCliAdapter (entrega no host)
│           ├── persistence/sqlite/              Event Store, missões
│           └── ai/ollama/                       conversa e planejamento
├── alien-web/                                  React 18 · TypeScript · Vite
│   └── src/{chat,timeline,panels/{terminal,diff,graph,preview,plan},approvals}
├── toca/                                       imagem do ambiente efêmero
│   ├── Dockerfile
│   ├── toca-entrypoint.sh
│   ├── opencode.base.json
│   └── templates/                             spring-boot-api, react-vite, python-cli
├── infra/
│   └── docker-compose.yml                     ollama-proxy, rede alien-net, pkg-proxy opcional
├── docs/
│   ├── alien-code-especificacao.pdf
│   └── pdf/                                   gerador deste documento
```

Camada	Escolha	Por quê
Backend	Java 21, Spring Boot 3, Spring AI, Spring WebSocket	Continuidade com a V1; virtual threads para SSE
Frontend	React 18, TypeScript, Vite	Mesma base da V1
Persistência	SQLite (Event Store + missões)	Local, sem servidor, durável
Containers	Docker Engine API via docker-java	Controle fino de limites e labels
Harness	opencode (modo servidor)	HTTP + SSE, Ollama, suporte AI-DLC e graphify
Metodologia	AI-DLC 2.x (harness opencode)	Perfis, portões, sensores, auditoria
Grafo	graphify (tree-sitter, MCP)	Local, determinístico, multi-repo
Modelos	Ollama no host (ex.: qwen3-coder, devstral, gpt-oss)	Abertos e locais
Testes	JUnit 5, Testcontainers, Vitest, Playwright	Unidade, integração com Toca real, e2e da timeline

application.yml (valores iniciais)

```
alien:
  model:
    chat: ollama/qwen3:14b
    coder: ollama/qwen3-coder:30b
  toca:
    image: alien/toca:0.1
    cpus: 4
    memory: 6g
    ttl: 60m
    network: allowlist          # none | allowlist
    parallel-tasks: 2
  halting:
    max-iterations: 8
    max-tokens: 400000
    task-timeout: 20m
    no-progress-limit: 3
  aidlc:
    mode: lite                  # full | lite
  graph:
    affected-depth: 2
```

14. Plano de entrega

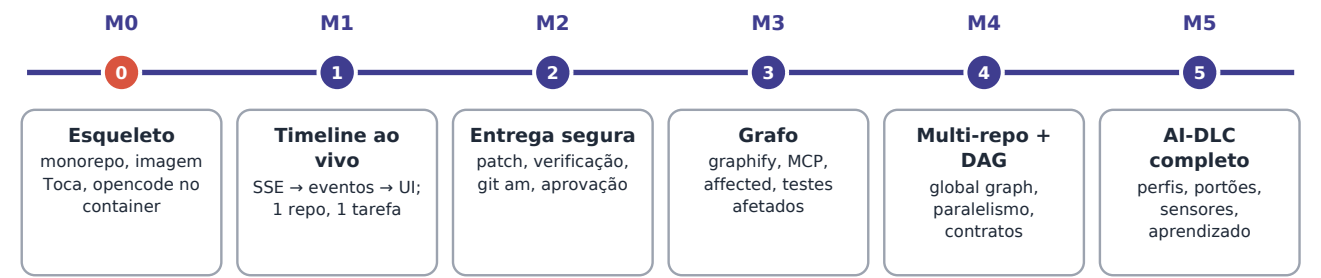


Figura 11. Marcos de entrega. Cada marco termina com algo demonstrável.

Marco	Critério de pronto
M0 · Esqueleto	Monorepo criado; imagem alien/toca sobe opencode serve; Alien Server cria e destrói a Toca; teste de integração com Testcontainers.
M1 · Timeline ao vivo	Missão com 1 repo e 1 tarefa; SSE do opencode vira eventos; UI mostra passos, tools e terminal; Parar funciona; reconexão com lastSeq.
M2 · Entrega segura	Pós-condições executadas pelo orquestrador; halting; aprovação de permissões; patch aplicado em branch local; gitleaks.
M3 · Grafo	graphify na Toca; MCP no opencode; testes afetados; aba Grafo; graph diff na entrega.
M4 · Multi-repo + DAG	Grafo global; planejador gera DAG; worktrees paralelas; contratos entre tarefas.
M5 · AI-DLC completo	Modo full com perfis e portões na UI; auditoria anexada; aprendizado com aprovação.

15. Riscos, decisões e glossário

15.1 Riscos

Risco	Impacto	Resposta
Modelos locais pequenos não sustentam tarefas longas ou o AI-DLC completo	Alto	Modo lite; tarefas pequenas no DAG; grafo via MCP para poupar contexto; modelo por missão.
Hardware do dev insuficiente (GPU/RAM) para modelo + builds	Alto	Paralelismo 1 por padrão; limites da Toca; Ollama pode apontar para outra máquina da rede.
Mudanças na API/eventos do opencode entre versões	Médio	Versão fixada; adaptador isolado; testes de contrato contra /doc.
Inferência do graphify errada (aresta INFERRED/AMBIGUOUS)	Médio	DAG exige aprovação humana; arestas AMBIGUOUS destacadas no plano.
Conflitos ao integrar worktrees paralelas	Médio	Paralelismo só entre tarefas sem aresta; conflito vira tarefa de integração.
Build sem rede falha por dependência nova	Médio	Proxy de pacotes com allowlist e aprovação.

15.2 Decisões em aberto

- Usar o SDK JS do opencode num sidecar Node ou consumir HTTP/SSE direto do Java (proposta: Java direto).
- Uma Toca por missão (proposta) ou Toca reutilizável por repositório com snapshot.
- Guardar o grafo global da missão na Toca (proposta) ou manter um global persistente no host entre missões.
- Abrir PR pelo host automaticamente ou só oferecer o comando (proposta: só oferecer).

15.3 Glossário

Termo	Significado
Missão	Unidade de trabalho pedida pelo usuário; tem plano, Toca, eventos e entrega.
Toca	Container efêmero de uma missão, onde o agente trabalha.
Tarefa	Nó do DAG; executada por uma sessão do opencode e verificada por pós-condições.
DAG	Grafo acíclico dirigido que define a ordem e o paralelismo das tarefas.
Harness	Programa que executa o loop do agente (aqui, o opencode).
Pós-condição	Verificação executável que decide se uma tarefa está correta.
Halting	Condição de parada do loop autocorretivo; ao atingir, escala ao humano.
Portão	Ponto em que o fluxo espera aprovação humana.
affected	Consulta do graphify que devolve o raio de impacto de um símbolo.
MCP	Model Context Protocol; forma padrão de expor ferramentas (como o grafo) ao agente.